

DP1 Peer Review Reflection

Github: <https://github.students.cs.ubc.ca/CPSC416-2025W-T2/mtahir03>

Implementation: 1dff2dc0c50d2a605fb90896c31d57280ec94ca (hash)

Location: projects/dp1/lab/src

Chosen Design:

<https://github.students.cs.ubc.ca/CPSC416-2025W-T2/lmcmulle/blob/main/design/dp1/design.md>

Original feedback: <https://github.students.cs.ubc.ca/CPSC416-2025W-T2/lmcmulle/issues/4>

My main critiques of the original design were regarding the failure in which backup sends heartbeats but does not respond to the primary. And further detail on when the primary will switch (from retrying on backup) to request view change from view server in such a scenario. I also asked for additional clarity regarding concurrency, but that critique was unfounded as we deal with a single-threaded application. Finally, I was curious about the discrepancy between freeze writes vs stop responding to requests specified when switching to a new primary (promotion protocol). Other than that I thought the design document was excellent and had everything needed to implement.

Failure Model:

—

Like I mentioned the lack of clarity on exactly how to deal with the backup failure in a case where it is alive and the primary does not respond. Specifically in terms of how long to retry, who retries, and when we should switch to asking the ViewServer to replace the backup caused issues. Some of my initial implementations ended up waiting indefinitely waiting for the backup server to respond.

Additionally there were a ton of issues with race conditions I was struggling with. One in particular was the issue in which primary sends a request to backup, and before the ack returns we update the view. Now the old primary receives the ack and is able to continue progressing until it receives the updated view information and rejects the ack. The prior backup is now the primary, so I presume this can only happen a few times/transition and is caused primarily by the fact that messages are not necessarily guaranteed to arrive in order of sending. This causes us to violate our condition of linearizability as we have inconsistent views.

Another issue I noticed was some ambiguity regarding “Self-demotion” which was mentioned in the design doc as a way to combat partitions. What exactly does a self-demotion look like? I had to come up with this, but some more clarity here would be nice. I.e. do we block while “self-demoting” what if we receive a heartbeat in between? What about race conditions where we send a self-demotion so the ViewServer updates the view but we do not see it and we receive the backup Ack.

Finally, the design didn't really consider cases in which we have multiple overlapping failures happening at once. (ie. partitions, timeouts, view changes, retries). Although I think you are able to handle this without thinking explicitly about all of the cases, it can lead to rare edge cases like the one mentioned above. I.e. what if one primary and backup server are partitioned and do not receive the view updates because of it?

There weren't any assumptions necessary that weren't in the original document.

Protocol

The original design did not have enough detail on primary-backup communication. Specifically, it wasn't specified what type of Acks would be sent between the two and what those acks would look like. Furthermore, what happens in failure scenarios? Does the primary keep retrying, what about the backup? This was alluded to but not fully elicited in your document.

The ideas of retries is also lacking a bit of detail. In general who is supposed to retry and is the retry interval constant?

Original Feedback Critique

I think the main issue in my original feedback was not considering what can happen in race conditions, specifically in scenarios in which multiple failures overlap and we have in-flight messages. This broke the assumption of linearizability and caused inconsistent view errors in my implementation. I would avoid missing it in the future by explicitly considering the interactions between multiple factors in my design, rather than treating them as individual issues. Of course, this is a learning lesson as naively I did not even consider such a case when originally reading the designs.